
Thinking Functionally in the JVM
Daniel Hinojosa


Reproducible Development Environments

The Problem

- "Works on my machine" is a team liability
- Reproducibility means the same tools and versions everywhere — no drift
- Some Java Tools come to mind
  - Nix
  - Docker
  - DevContainers

Nix

- Nix is a powerful package manager and build system, known for its unique approach to package and configuration management.
- It’s primarily used in Unix-like systems and has several distinctive features:
- **Functional Package Management**
- Reliable and Reproducible
- Isolated Environments
- Rollbacks and Atomic Upgrades
- Declarative Configuration
- Multi-user Profiles

```nix
{ pkgs ? import <nixpkgs> {} }:
pkgs.mkShell {
  buildInputs = [ pkgs.jdk21 ];
}
```

Docker

- Lightweight containerization platform
- Packages applications with all dependencies
- Runs isolated environments using OS-level virtualization
- Uses layered images built from Dockerfiles
- Widely used for reproducible builds and deployments

dockerfile

```
FROM openjdk:21
WORKDIR /app
CMD javac HelloWorld.java &&
    java HelloWorld
```

Dev Containers

- Lightweight containerization platform
- Packages applications with all dependencies
- Runs isolated environments using OS-level virtualization
- Uses layered images built from Dockerfiles
- Widely used for reproducible builds and deployments

SDKMan Intro

The Software Development Kit Manager

Meet **SDKMAN!** – your reliable companion for effortlessly managing multiple Software Development Kits on Unix systems. Imagine having different versions of SDKs and needing a stress-free way to switch between them. SDKMAN! steps in with its easy-to-use Command Line Interface (CLI) and API. Formerly known as GVM, the Groovy enVironment Manager, SDKMAN! draws inspiration from familiar tools like apt, pip, RVM, and rbenv and even Git. Think of it as your helpful toolkit friend, ready to streamline SDK management for you. 🛠️

Get Started Now!

Go on then, paste and run the following in a terminal:

```
curl -s "https://get.sdkman.io" | bash
```

SDKMan

- Lightweight JVM version switcher — no containers required
  - `sdk list` - List all Tools
  - `sdk list java` — List all Java Versions and their identifiers
  - `sdk install java <identifier>` — Install a Specific Java
  - `sdk use java <identifier>` — Use a specific Java that has already been installed
- Pin a project's Java version with `.sdkmanrc` in your project, then run `sdk install`

```properties
java=21.0.2-tem
maven=3.9.6
```


Functional Java

Java's Functional Toolkit

- `java.util.function` ships `Function`, `Predicate`, `Supplier`, and `Consumer`
- These are the building blocks for functional-style code — no third-party libs needed

`Function<T, R>`

- Transforms a value of type `T` into a value of type `R`
- Compose pipelines with `andThen` and `compose`

```java
Function<String, Integer> length =
    value -> value.length();
```

`Predicate<T>`

- Tests a condition — returns `true` or `false`
- Combine predicates with `and`, `or`, `negate`

```java
Predicate<String> notEmpty = s -> !s.isEmpty();
```

`Supplier<T>`

- Produces a value lazily — no input, one output
- Useful for deferred or optional computation

```java
Supplier<List<String>> fresh =
    () -> new ArrayList<>();
```

`Consumer<T>`

- Performs a side effect on a value — no return
- Use for logging, printing, or persisting

```java
Consumer<String> print =
    value -> System.out.println(value);
```

Method References

- A method reference is a shorter lambda
- Use it when the lambda only calls one existing method or constructor

```java
Function<String, Integer> length =
    value -> value.length();

Function<String, Integer> same =
    String::length;
```

Constructor References

- `ArrayList::new` means "call the `ArrayList` constructor"
- The functional interface decides the shape: no arguments, one result

```java
Supplier<List<String>> fresh =
    () -> new ArrayList<>();

Supplier<List<String>> same =
    ArrayList::new;
```

Static Method References

- `ClassName::method` calls a static method
- The lambda arguments become the method arguments

```java
Function<String, Integer> parse =
    value -> Integer.parseInt(value);

Function<String, Integer> same =
    Integer::parseInt;
```

Instance Method References

- `instance::method` calls a method on a specific object
- The lambda argument is passed to that object

```java
Consumer<String> print =
    value -> System.out.println(value);

Consumer<String> same =
    System.out::println;
```

Method Reference Forms

Form	Reads As	Lambda Shape
`ClassName::staticMethod`	call a static method	`x → ClassName.staticMethod(x)`
`instance::method`	call a method on this object	`x → instance.method(x)`
`ClassName::instanceMethod`	call a method on the input object	`x → x.instanceMethod()`
`ClassName::new`	call a constructor	`() → new ClassName()`

Use the Clearest Form

- Lambdas make the inputs visible
- Method references remove noise once the shape is clear

```java
Predicate<String> notEmpty =
    value -> !value.isEmpty();

Function<String, String> trim =
    String::trim;
```

Functional Design Principles

Referential Transparency

- Replace any function call with its return value and the program behaves the same
- No hidden state, no side effects — the same input always yields the same output

```java
int add(int a, int b) { return a + b; }
```

- Compare the above to the following, which is **not referentially transparent** we cannot replace the call with its result

```java
int addAndPrint(int a, int b) {
    int result = a + b;
    System.out.println(result); // IO changes the world
    return result;
}
```

Single Responsibility Principle

- Each function does exactly one thing
- Small, focused functions are easier to test, name, and compose
- Violation — one method **fetches**, **validates**, and **saves**:

```java
void processUser(User u) {
    var data = db.fetch(u.id());     (1)
    if (data == null) throw ...;     (2)
    db.save(data.activate()); (3)
}
```

(1) Fetching the data
(2) Validating the data
(3) Taking the data, activating it, and saving it

- Cleaned up — each function has one job:

```java
var data   = fetch(u.id());
var valid  = validate(data);
db.save(data.activate());
```

What Is a Side Effect?

- A side effect is any observable interaction beyond returning a value
- The method changes the world, reads from the world, or depends on a hidden state
- The following shows no side effect:

```java
int add(int a, int b) {
    return a + b;
}
```

- Here is an example of printing to the screen, which is a side effect:

```java
int addAndPrint(int a, int b) {
    int result = a + b;
    System.out.println(result);
    return result;
}
```

Common Java Side Effects

Side Effect	Example
Console I/O	`System.out.println(value)`
Updating state	`this.total += amount`
Files and databases	`Files.write(…)`, `repository.save(…)`
Network calls	`httpClient.send(request, handler)`
Time	`LocalDate.now()`, `Instant.now()`
Randomness	`UUID.randomUUID()`, `Random.nextInt()`
Exceptions	`throw new IllegalStateException(…)`

Separate Side Effects

- Push I/O and mutation to the edges of the program
- Pure core logic stays testable when it doesn't touch the outside world

  Before:

```java
void printInvoice(Invoice invoice) {
    LocalDate today = LocalDate.now();
    Money total = invoice.totalFor(today);
    System.out.println(total);
}
```

  After:

```java
Money totalFor(Invoice invoice, LocalDate today) {
    return invoice.totalFor(today);
}

void printTotal(Money total) {
    System.out.println(total);
}
```

Separate Side Effects: Inject Time

- If a method needs today's date, make the dependency explicit
- `Supplier<LocalDate>` keeps time replaceable in tests

```java
class InvoiceService {
    private final Supplier<LocalDate> today;

    protected InvoiceService(Supplier<LocalDate> today) {
        this.today = today;
    }

    static InvoiceService of() {
        return new InvoiceService(() -> LocalDate.now());
    }

    Money totalFor(Invoice invoice) {
        return invoice.totalFor(today.get());
    }
}
```

Command Query Separation

- A method either changes the state (command) or returns a value (query) — not both
- Mixing the two makes behavior hard to predict and test

Command and Query Mixed

- This method changes state and returns a value
- The caller has to know that asking for the result also mutates the object

```java
class Cart {
    private final List<Item> items = new ArrayList<>();

    Money addItem(Item item) {
        items.add(item);
        return total();
    }

    Money total() {
        return items.stream()
            .map(Item::price)
            .reduce(Money.ZERO, Money::add);
    }
}
```

Command and Query Separated

- Command: `addItem` has one purpose — change the cart
- Query: `total()` has one job — calculate and return the total

```java
class Cart {
    private final List<Item> items = new ArrayList<>();

    void addItem(Item item) {
        items.add(item);
    }

    Money total() {
        return items.stream()
            .map(Item::price)
            .reduce(Money.ZERO, Money::add);
    }
}
```

Functional Method Shape

- Prefer explicit inputs and meaningful return values
- A small method either returns a plain value or a value in a context

```java
Account mergeBalances(Account left, Account right) {
    return left.withBalance(
        left.balance().add(right.balance())
    );
}

Optional<Account> findAccount(AccountId id) {
    return repository.findById(id);
}
```

- The return type tells the caller what kind of result to expect
- `Account` means "you get an account"
- `Optional<Account>` means "maybe"

Small Functions, Large Programs

- Complex behavior emerges from composing small, focused functions
- The size of a program is no reason for individual functions to grow large


Immutable Data

Why Immutability?

- Immutable values cannot change after creation — no mutation surprises
- Thread-safe by default; safe to share, cache, and pass freely

Records

- Java `record` declares immutable data with minimal boilerplate
- Compiler generates constructor, accessors, `equals`, `hashCode`, and `toString`

```java
record Point(int x, int y) {}
var p = new Point(1, 2);
```

How Do You Update a Record?

- Records are immutable — construct a new one with the changed value

```java
Point newLoc = new Point(oldLoc.x() * 2, oldLoc.y(), oldLoc.z());
```

- Add `with`-style methods to make it readable and chainable

```java
record Point(int x, int y, int z) {
    Point withX(int x) { return new Point(x, y, z); }
    Point withY(int y) { return new Point(x, y, z); }
    Point withZ(int z) { return new Point(x, y, z); }
}
```

```java
Point newLoc  = oldLoc.withX(oldLoc.x() * 2);
Point nextLoc = newLoc.withY(newLoc.y() * 2)
                      .withZ(newLoc.z() * 2);
```

Derived Record Creation (Preview)

- JEP 468 proposes `with` expressions, the compiler generates the copy constructors for you
- No more handwritten `withX` / `withY` boilerplate

JEP 468 preview syntax

```java
Point newLoc  = oldLoc with { x = oldLoc.x() * 2; }
Point nextLoc = newLoc with { y *= 2; z *= 2; }
```

Targeted for preview in a future JDK release — JEP 468

Value Objects

- A value object represents a domain value, not an object identity
- Equality is based on its contents: `Money.of(100)` equals another `Money.of(100)`
- Good value objects make invalid or confusing code harder to write

Primitive Values Lose Meaning

- `double` tells us the shape, not the meaning
- The compiler cannot tell kilowatts from megawatt-hours

```java
double loadKw = 1.2;
double energyMwh = 24.2;

double sumKw = loadKw + energyMwh;
```

- The code compiles, but the math is nonsense

Value Objects Keep Meaning

- The type system carries the domain meaning
- The compiler recognizes mistakes before production does

```java
record Power(double kilowatts) {
    Power plus(Power other) {
        return new Power(kilowatts + other.kilowatts);
    }
}

record Energy(double megawattHours) {}

Power load = new Power(1.2);
Energy energy = new Energy(24.2);

Power sum = load.plus(energy);
```

- The last line does not compile: `Energy` is not `Power`

Prefer Value Objects

- Lower cognitive load: names and types explain what the value is
- Better design: illegal combinations do not look like ordinary math
- Easier refactoring: behavior can move next to the value it protects

Value Objects and Project Valhalla

- Project Valhalla introduces value types — flat, identity-free, no heap overhead
- Designed for small immutable data that are currently `boxed` as objects
- Here is an example of `Money` as a Valhalla `value` object, expected to be in preview in JDK 27

```java
value class Money(long cents) {}
```

JEP 401, which bring value objects to Java, is now listed as a preview feature.


Streams and Laziness

Lazy vs. Eager Evaluation

- **Eager**: evaluates immediately
  - Collections, Loops
- **Lazy**: evaluates on demand
  - Java Streams, `Supplier`

Delayed Computation

- Lazy evaluation defers work until the result is needed
- Avoids computing things that may never be used

```java
Supplier<LocalDateTime> now =
    () -> LocalDateTime.now();
```

- Time is not read until `get()` runs

```java
LocalDateTime startedAt = now.get();
```

Memoization

- The previous example is not memoized, each call to `get()` recomputes the value
- Memoization caches the first result and reuses it on subsequent calls
- We can achieve Memoization with a new feature in Java, `LazyConstant`

```java
LazyConstant<LocalDateTime> localDateTimeLazyConstant =
   LazyConstant.of(() -> LocalDateTime.now());
```

- Given the above and given the time is `2026-07-04T10:26:51-07:00`, using `get()` will always return the same date and time because it is *memoized*

```java
localDateTimeLazyConstant.get();
```

`LazyConstant` is a Preview API in JEP 526

Streams are Lazy

- Streams are inherently lazy, nothing will execute until you perform a *terminal operation*
- `filter`, `map`, `flatMap` are intermediate operations
- Terminal operations include `collect`, `findFirst`, `reduce`, `toList`
- The following just returns a pipeline object. In OpenJDK it is called a `ReferencePipeline`

```java
Stream.of(1, 2, 3).filter(n -> n > 1);
```

Pipelines

In the following example

- `stream()` creates the stream
- `filter()` and `map` are intermediate operations
- `toList()` is a terminal operation, the stream has materialized

```java
list.stream()
    .filter(s -> !s.isEmpty())
    .map(String::toUpperCase)
    .toList();
```


Function Composition

Composing Functions

- Composition wires the output of one function into the input of another
- Build complex transformations from small, focused pieces
- No new classes are required

andThen

- `f.andThen(g)` applies `f` first, then passes the result to `g`
- Reads left to right — matches the direction data flows

```java
Function<String, String> trim  = String::trim;
Function<String, String> upper = String::toUpperCase;
Function<String, String> clean = trim.andThen(upper);
```

- Running `clean` will return `"HELLO"`

```java
clean.apply("Hello    ");
```

andThen with Mixed Types

- The output type of one function must match the input type of the next
- `String → String → Integer → Integer`

```java
Function<String, String> trim =
    String::trim;

Function<String, Integer> parseInt =
    Integer::parseInt;

Function<Integer, Integer> mult3 =
    n -> n * 3;

Function<String, Integer> pipeline =
    trim.andThen(parseInt).andThen(mult3);
```

- Applying the function will provide `42`

```java
pipeline.apply("  14  ");
```

`compose`

- `compose` chains functions in mathematical order
- `f.compose(g)` applies `g` first, then `f`

  $y = f(g(x))$
- Given three functions, `f`, `g`, and `h`

  $y = f(g(h(x)))$
- Useful when building from the outside in, therefore using our `andThen` example with `compose` it will look like the following:

```java
Function<String, Integer> pipeline =
    mult3.compose(parseInt).compose(trim);
```

- Applying the function will provide `42`

```java
pipeline.apply("  14  ");
```


Algebraic Data Types

What Are Algebraic Data Types?

- A closed set of types — either a product (AND) or a sum (OR)
- The type system enforces exhaustive handling; no case is silently ignored

Product Types

- A product type combines fields together
- Read it as AND: a `Planet` has a name, radius, and mass

```java
record Planet(
    String name,
    double radius,
    double mass
) {}
```

Sum Types

- A sum type chooses one case from a closed set
- Read it as OR: a `Shape` is a `Circle`, `Rectangle`, or `Triangle`

```java
sealed interface Shape
    permits Circle, Rectangle, Triangle {}

record Circle(double radius) implements Shape {}
record Rectangle(double length, double height) implements Shape {}
record Triangle(double base, double height) implements Shape {}
```

Use `enum` for Same Shape

- Use an `enum` when each case is semantically the same kind of thing
- Each planet has the same fields: `radius` and `mass`

```java
enum Planet {
    MERCURY(2_439.7, 3.3011e23),
    VENUS(6_051.8, 4.8675e24),
    EARTH(6_371.0, 5.97237e24);

    private final double radius;
    private final double mass;

    Planet(double radius, double mass) {
        this.radius = radius;
        this.mass = mass;
    }
}
```

Use Sealed Types for Different Shapes

- Use sealed types when each case has different semantics
- Circle, Rectangle, and Triangle are all shapes, but they are not built the same way
  - Circle is defined by its `radius`
  - Rectangle is defined by its `height` and `width`
  - Triangle is defined by its `height` and `base`

Describe First, Interpret Later

- Algebraic Data Types describe the data first
- What we do with that data is left to interpretation *after defining the data*
- This is the beginning of the interpreter pattern

```java
sealed interface Shape
    permits Circle, Rectangle, Triangle {}
```

- Given our shape, we can interpret it as an SVG, drawing onto a UI, or describing it as JSON.

```java
String drawSvg(Shape shape) { ... }
void drawCanvas(Shape shape, Canvas canvas) { ... }
String toJson(Shape shape) { ... }
```

Commands

- One sealed variant per action — no ambiguity about what is being requested
- Pattern match to handle each command explicitly

```java
sealed interface Command
    permits Deposit, Withdraw, Transfer {}
```

Results and Errors

- Model success and failure as data — not exceptions
- The return type signals what can go wrong

```java
sealed interface Result<T>
    permits Success, Failure {}
```

Collections

- A list is either empty or has a head and a tail — a sum type
- Pattern match on structure the same way you match on domain types

Language and the Interpreter Pattern

- Define vocabulary of operations as sealed types — a small language
- Separate description from execution; an interpreter runs the program later

Demo: Sealed Types and Interpreters in Java

Let's view some examples of `sealed` types and how they are interpreted in Java.


Errors as Data

The Problem with Exceptions

- Exceptions break composition — they escape the normal return path
- Callers can’t tell from the signature that a method might fail

Homemade `sealed` Results

- In the previous demo, we used Algebraic Data Types to model results and errors.

Creating a `sealed Result`

```java
sealed interface OrderResult {
    record Success(OrderId id) implements OrderResult {}
    record Failure(OrderError error) implements OrderResult {}
}
```

Creating a `sealed Error`

```java
sealed interface OrderError {
    record CustomerNotFound(CustomerId id) implements OrderError {}
    record ProductNotFound(ProductId id) implements OrderError {}
}
```

- Functional code often makes expected errors explicit in the return type
- This is making errors as data, **not things that are thrown out and disruptive**, part of a contract that we can reason about

`Optional` / `Option`

- Represents a value that may or may not be present
- Forces callers to handle absence explicitly — no null leaks
- Comes in Java, `Optional.of(…)`, in Scala it is `Option`
- Kotlin often uses nullable type `T?`

```java
Optional<User> findUser(int id) { ... }
```

`Either<L, R>`

- Callers must handle both paths; nothing is hidden
- Semantically, it means that something can go specifically wrong.
- Left generally wraps around any generic type of your choice
- Has two cases: `Right` which is a success: `Left` which is a failure
- Not available in Java, though you can create your own or use Vavr
- In Kotlin, you can use a library, Arrow
- Available in the Scala Standard Library

`Try<T>`

- Wraps a computation that might throw and converts exceptions to values
- Composable — chain operations without `try/catch`
- Semantically, it means that something can go generically wrong
- Similar to `Either`, but `Failure` always wraps a `Throwable`
- Has two children: `Success` and `Failure`
- Not available in Java, though you can create your own or use Vavr
- In Kotlin, it is not available, though you can use `Either` with Arrow
- Available in the Scala Standard Library

`Validated`

- Accumulates multiple errors instead of short-circuiting on the first
- Ideal for form validation, config parsing, or input checking
- Has two children: `Valid`, and `Invalid`
- Not available in Java, though you can create your own or use Vavr
- In Kotlin, you can use a library, Arrow
- Not available Scala Standard Library, but available in many libraries

Compare to Exceptions

- Exceptions: implicit, non-composable, invisible in type signatures
- Error types: explicit, composable, visible at every call site

Returning Composable Values

- `Optional`, `Validated`, and `CompletableFuture` all support `map` and `flatMap`, although for `CompletableFuture`, they are called `thenApply` and and `thenCompose`.
- Errors propagate through the chain without `try/catch` boilerplate

Demo: Creating an `Either`

Let’s watch as I use an Agent to convert the HexArch application into something that uses a custom `Either`.

Functional Behaviors

`map`

- Applies a function to the value inside a container — the container shape is preserved
- Works on `Optional`, `Stream`, `CompletableFuture`

```java
Optional.of("hello").map(String::toUpperCase);
```

`flatMap`

- Like `map`, but the function returns a container — avoids nesting
- The monadic pattern: `flatMap`, `flatMap`, …, `map`

```java
findUser(id)
    .flatMap(u -> findOrder(u.id()))
    .flatMap(o -> findItem(o.itemId()))
    .map(Item::name);
```

In-Channel Errors

- `flatMap` short-circuits on `Empty` / `Left` / `Failure`
- Error handling is structural
- No explicit `if` or `try`/`catch` checks needed

The following will return `Optional.of(6)`

```java
var o1 = Optional.of(2);
var o2 = Optional.of(4);
var result = o1.flatMap(x -> o2.map(y -> x + y));
```

- Here is where the short-circuit will happen

The following will return `Optional.empty()`

```java
var o1 = Optional.of(2);
var o2 = Optional.<Integer>empty();
var result = o1.flatMap(x -> o2.map(y -> x + y));
```

With `Optional`, `Stream` and `CompletableFuture`

- `Optional` and `Stream` both have `flatMap`
- `CompletableFuture.thenCompose` is `flatMap` for async computations

`filter`

- Drops values that don’t satisfy a predicate; returns an empty container if none pass

```java
stream.filter(n -> n > 0)
```

reduce

- Folds a collection into a single value using an identity and a combining function
- The following will return a `6`

```java
Stream.of(1, 2, 3).reduce(0, Integer::sum);
```

When Short-Circuiting Is Wrong

- `flatMap` stops at the first failure
- Sometimes we want every check to run
- Example: validation should report all problems at once

```java
Validated<Name> name =
    validateName(input.name());

Validated<Email> email =
    validateEmail(input.email());

Validated<Age> age =
    validateAge(input.age());
```

A Validation Harness

- A harness applies independent checks
- Each check contributes success or errors
- The final result keeps all collected errors

```java
Validated<Customer> result =
    Validated.combine(name, email, age)
        .map(Customer::new);
```

- This does not stop after `name` fails, `email` and `age` will still run.

Lab: Functional Behaviors

We will run a few standard functional operations, you will create your own `Optional` chain. Next we will run a `CompletableFuture`. Finally, we will create our own `Validated` harness, so we can verify that all validations run.


Java Thoughts and Patterns

Object Behavior

- `equals` and `hashCode` must be consistent — records handle this automatically
- `Comparable` for natural order; `Comparator` for flexible, external ordering

```java
Comparator<Person> byAge = Comparator.comparing(Person::age);
```

Structured Concurrency

- `StructuredTaskScope` runs subtasks in a structured tree — cancel on first failure
- Composable concurrent work without manual thread wiring

```java
try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
    var user = scope.fork(() -> findUser());
    var invoices = scope.fork(() -> findInvoices());
    scope.join();
    return new Response(user, invoices);
}
```

Vavr Validation

- Vavr's `Validation` accumulates all errors before returning — no early exit
- Compose individual validators with `combine` into a single result
- `.ap` applies the constructor only when all validations are valid

```java
public Validation<Seq<String>, User> validateUser(
    String name, String email) {
    return Validation
        .combine(
            validateName(name),
            validateEmail(email))
        .ap(User::new);
}
```

Vavr calls this operation `.ap`

Functional Operations Recap

- `flatMap` — sequence dependent computations; short-circuits on failure
- `map` — transform a value without unwrapping the container
- `reduce` — collapse a collection into one value
- `ap` — combine independent values while keeping collected errors


About Scala

Scala on the JVM

- Runs on the JVM and interoperates freely with Java libraries
- Statically typed with a substantially richer type system than Java

What Scala Adds to Java

- Case classes
- Pattern matching
- Extension methods
- Typeclasses
- Higher-kinded types
- Opaque types
- Open classes
- Union and intersection types

Why It Matters Here

- Java can model many functional ideas directly
- Scala can abstract over those ideas more naturally
- The rest of the workshop asks: what becomes easier when the type system can express more?

Higher-Kinded Types

- Scala can abstract over `F[A]`
- Java can abstract over `A`, but not over the shape `F[_]`
- This is the key move behind reusable effectful programs

When to Consider Scala

- When the Java type system cannot express your domain clearly
- When repeated Java patterns point to a missing abstraction
- When effect systems, typeclasses, or HKTs make the design simpler

The Trade-off

- Greater expressiveness comes with a steeper learning curve
- Team familiarity and hiring pool matter as much as language power


Scala Context Parameters

Why This Syntax Matters

- Some values are used across many method calls
- Scala uses contextual values and parameters for that wiring
- The compiler can pass the right value when the type matches

`given`

- A `given` defines a contextual value
- Think of it as a value the compiler is allowed to find

```scala
final case class AppConfig(prefix: String)

given AppConfig =
  AppConfig("[workshop]")
```

`using`

- A `using` parameter asks the compiler for a contextual value
- The method states what context it needs

```scala
def log(message: String)(using config: AppConfig): Unit =
  println(s"${config.prefix} $message")

log("starting")
```

Summoning

- `summon` retrieves a contextual value explicitly
- Useful when you want to name the context inside a method

```scala
def prefix(using AppConfig): String =
  summon[AppConfig].prefix
```

Typeclasses use this same mechanism; the next chapter gives the pattern a name.


Typeclasses

What Is a Typeclass?

- A typeclass defines a capability that a type can satisfy from the outside
- Behavior is decoupled from data: open to extension without inheritance
- The type stays simple; the behavior is supplied separately

Why Java Developers Should Care

- `Comparator<T>` already feels typeclass-shaped
- It adds ordering without changing the original type
- Scala generalizes this pattern and lets the compiler find the right behavior

Show

- Converts a value to a human-readable string
- Defined outside the class: no coupling to the data type

```scala
given Show[User] with
  def show(user: User): String =
    s"User(${user.name})"
```

- The same instance can be written in shorthand

```scala
given Show[User] = user => s"User(${user.name})"
```

Eq

- Type-safe equality: can't accidentally compare incompatible types
- Defined separately from `equals`; compiler enforces it

```scala
given Eq[Money] with
  def eqv(left: Money, right: Money): Boolean =
    left.cents == right.cents
```

- The same instance can be written in shorthand

```scala
given Eq[Money] =
  (left, right) => left.cents == right.cents
```

`Order` and `Comparator`

- `Order` is a typeclass for total ordering

```scala
given Order[Money] with
  def compare(left: Money, right: Money): Int =
    left.cents.compare(right.cents)
```

- Java's `Comparator` is the closest analog: explicit, not inherited
- Java had the right abstraction idea all along

```java
Comparator<Money> byAmount = Comparator.comparingLong(Money::cents);
```

Java’s Path to Typeclasses

- OpenJDK experiments have explored typeclass-like interface capabilities
- This is not ordinary Java today
- The direction matters: behavior can be selected by type, not inheritance

Separating Behavior from Data

- Add new operations to existing types without touching them
- Makes code open-closed at the data layer, not just the object layer
- This is the bridge from Java patterns to Scala typeclasses


Higher-Kinded Types

`F[A]`

- A higher-kinded type takes another type as a parameter: `F[_]`
- Write code that works for any container: `Option`, `List`, `Future`

The Shape Is the Difference

- `Future[User]`
- `Option[User]`
- `Either[Throwable, User]`
- The payload is still `User`
- The context is what changes

One Program

- **Before:** Repeat the same logic for each effect

```scala
def loadFuture(id: UserId): Future[User]

def loadOption(id: UserId): Option[User]

type MyEither[A] = Either[Throwable, A]
def loadEither(id: UserId): MyEither[User]
```

- **After:** Abstract over the context

```scala
def load[F[_]](
    id: UserId,
    repository: UserRepository[F]
): F[User]
```

Type Lambdas

- Sometimes the effect has more than one type parameter
- Fix one side, then abstract over the remaining `A`

```scala
type MyEither = [A] =>> Either[Throwable, A]

def load[F[_]](id: UserId): F[User]
```

Reusable Programs

- Test with one effect
- Deploy with another effect
- Keep the business program the same

Java's Limitation

- Java generics cannot express `F[_]`: only `F<A>` with a concrete `F`
- Workarounds exist (Vavr, higher-kinded emulation) but are verbose and incomplete


Functional Abstraction Typeclasses

Monoid

- A type with a binary `combine` operation and an identity (zero) element
- Strings, lists, and integers under addition are all monoids

```scala
trait Monoid[A]:
  def empty: A
  def combine(left: A, right: A): A

given Monoid[String] with
  def empty: String = ""
  def combine(left: String, right: String): String =
    left + right

summon[Monoid[String]].combine("hello ", "world")
```

`Monoid` in Java

- Java can express the same capability as an interface

```java
interface Monoid<T> {
    T empty();
    T combine(T left, T right);
}
```

- The instance is an object instead of a compiler-selected `given`

```java
Monoid<String> stringMonoid = new Monoid<>() {
    public String empty() { return ""; }
    public String combine(String left, String right) {
        return left + right;
    }
};

stringMonoid.combine("hello ", "world");
```

There is a Brian Goetz video on Growing a Language and using Typeclasses

Experimental Java Witness

- A witness is evidence that a type has a capability

```java
__witness Monoid<String> STRING_MONOID = new Monoid<>() {
    public String empty() { return ""; }

    public String combine(String left, String right) {
        return left + right;
    }
};
```

- The witness can be looked up by type

```java
Monoid<String> monoid = Monoid<String>.__witness;

monoid.combine("hello ", "world");
```

⚠ **This is not Java today.** OpenJDK Valhalla explored this as experimental typeclass support; the syntax and feature are not settled.

Functor

- Any type that supports `map` over its contents while preserving structure
- Laws: mapping the identity function changes nothing; composition holds
- Remember `map`: this is the abstraction behind it

```scala
trait Functor[F[_]]:
  def map[A, B](fa: F[A])(f: A => B): F[B]

Functor[Option].map(Some("ada"))(_.toUpperCase)
```

Applicative

- Applies a wrapped function to a wrapped value: `F[A ⇒ B]` to `F[A]`
- Enables independent effects to run without sequencing
- Remember `Validation`: this is how independent checks combine

```scala
trait Applicative[F[_]] extends Functor[F]:
  def pure[A](value: A): F[A]
  def ap[A, B](ff: F[A => B])(fa: F[A]): F[B]
```

`Applicative` with `ValidatedNel`

- Combine independent validations

```scala
Applicative[[A] =>> ValidatedNel[String, A]]
  .map3(
    validateName(""),
    validateSsn("123-44-32013"),
    validateDateOfBirth(LocalDate.of(1982, 10, 11))
  )((name, ssn, dateOfBirth) =>
    Person(name, ssn, dateOfBirth)
  )
```

- This returns both validation errors

```scala
Invalid(NonEmptyList(
  "Name cannot be blank",
  "SSN is not valid"
))
```

`ValidatedNel[String, A]` accumulates errors in a non-empty list.

Monad

- A `Functor` with `flatMap`: chains dependent computations
- Remember `flatMap`: this is the abstraction behind it

For-Comprehensions

- Scala syntax for nested `flatMap` and a final `map`
- Everything to the right of ← must be the same context type

```scala
for {
  x <- Option(3)
  y <- Option(2)
  z <- Option(1)
} yield x + y + z
```

These Two Are Equal

- Nested `flatMap` and `map`

```scala
Option(3)
  .flatMap(x =>
    Option(2)
      .flatMap(y =>
        Option(1)
          .map(z => x + y + z)))
```

- The same expression as a `for`-comprehension

```scala
for {
  x <- Option(3)
  y <- Option(2)
  z <- Option(1)
} yield x + y + z
```

In-Channel Errors

- Monadic `flatMap` short-circuits on the first failure
- Use `Applicative` when all errors should accumulate instead

Foldable

- Folds a structure into a summary value
- This is the abstraction behind `reduce`
- It does not preserve the original shape

```scala
trait Foldable[F[_]]:
  def foldLeft[A, B](fa: F[A], start: B)
      (f: (B, A) => B): B

Foldable[List].foldLeft(List(1, 2, 3), 0)(_ + _)
```

Traverse

- Flips a collection of effects into an effect of a collection
- `List[Future[T]]` to `Future[List[T]]`: sequence without manual loops
- It walks a structure while preserving the structure

```scala
trait Traverse[F[_]] extends Functor[F], Foldable[F]:
  def traverse[G[_]: Applicative, A, B](fa: F[A])
      (f: A => G[B]): G[F[B]]

List(1, 2, 3).traverse(findUser)
```

`Foldable` vs. `Traverse`

Typeclass	Question	Shape
`Foldable`	How do I summarize this?	`List[A]` to `B`
`Traverse`	How do I walk this with an effect?	`List[F[A]]` to `F[List[A]]`


Effectful Programs

What Is an Effect?

- An effect is a description of a computation that interacts with the outside world
- Effects are values: they describe what to do, not do it immediately

`Future`

- Starts running immediately: eager, and hard to reason about in composition
- Error handling is callback-based and easy to accidentally drop

A Toy `IO`

- This is a teaching model, not Cats Effect internals
- The important idea: describe now, run later

```scala
sealed trait IO[A]

object IO:
  final case class Pure[A](value: A) extends IO[A]
  final case class Delay[A](thunk: () => A) extends IO[A]
  final case class FlatMap[A, B](
      source: IO[A],
      next: A => IO[B]
  ) extends IO[B]
```

`Delay` should look familiar: this is the same idea as `LazyConstant`.

Programs as Values

- `IO[A]` is a value that describes how to produce an `A`
- Nothing happens until an interpreter runs it

```scala
val greeting: IO[String] =
  IO.Delay(() => "Hello")
```

Cats Effect `IO`

- Describes a computation without executing it: lazy and composable
- Nothing runs until you call the interpreter at the edge of the world

```scala
val program: IO[Unit] =
  IO.println("Hello") >> IO.println("World")
```

Context Bounds

- Context bounds are compact `using` syntax
- `F[_]: Monad` means "there is a `Monad[F]` available"

```scala
def program[F[_]: Monad](value: Int): F[Int] =
  Monad[F].pure(value)
```

The Translation

- The compact syntax hides a `using` parameter

```scala
def program[F[_]: Monad](value: Int): F[Int]

def program[F[_]](value: Int)(using Monad[F]): F[Int]
```

MonadThrow

- A monad with in-channel error capability
- We already saw this idea: *errors travel in the returned value*
- Use it as a constraint when the program can fail

```scala
def requirePositive[F[_]: MonadThrow](value: Int): F[Int] =
  if (value > 0) then value.pure[F]
  else MonadThrow[F].raiseError(
    IllegalArgumentException("value must be positive")
  )
```

`F[_]`: Abstract Effects

- Write logic against `F[_]` which let's us swap the runtime without changing the program
- Enables testability, mock effects, and multiple backends

```scala
def createOrder[F[_]: MonadThrow](
    command: CreateOrder
): F[OrderId]
```

Clocks and Time

- Direct time reads are side effects

```scala
val startedAt: Long =
  System.currentTimeMillis()
```

- Cats Effect makes time an effectful capability

```scala
def measure[F[_]: Clock, A](program: F[A])
    (using Monad[F]): F[(A, FiniteDuration)] =
  for {
    start <- Clock[F].monotonic
    value <- program
    end <- Clock[F].monotonic
  } yield (value, end - start)
```

Java Time as a Capability

- `cats-effect-time` provides Java Time operations as typeclass capabilities

```scala
import io.chrisdavenport.cats.effect.time.JavaTime

def createdAt[F[_]: JavaTime]: F[Instant] =
  JavaTime[F].getInstant
```

- Time stays replaceable in tests

> ℹ Cats Effect `Clock` provides monotonic and real time. `cats-effect-time` adds Java Time values such as `Instant`.

Using Java Time

- **Unit Test:** provide a fixed `Clock[IO]`

```scala
val fixed = Instant.parse("2026-01-01T00:00:00Z")

given Clock[IO] with
  def monotonic: IO[FiniteDuration] =
    IO.pure(0.seconds)

  def realTime: IO[FiniteDuration] =
    IO.pure(fixed.toEpochMilli.millis)

createdAt[IO]
```

- **Integration Test:** use the real runtime capability

```scala
import io.chrisdavenport.cats.effect.time.implicits._

createdAt[IO]
```

Swapping Effects in Tests

- The program asks for capabilities
- The test supplies a different interpreter
- The business logic does not change

```scala
val production: IO[OrderId] =
  createOrder[IO](command)

type TestEffect[A] = State[TestWorld, A]

val test: State[TestWorld, OrderId] =
  createOrder[TestEffect](command)
```

Programs vs. Values

- A program is just a value until interpreted: it can be inspected, composed, retried
- Separating description from execution is the foundation of effect systems


Programs and Interpreters

Data

- Domain values, commands, events, and errors
- These are the things we describe
- They carry meaning without running anything

Algebra

- Capabilities, ports, and business vocabulary
- What operations can this program ask for?

```scala
trait Orders[F[_]]:
  def create(command: CreateOrder): F[OrderId]

trait Payments[F[_]]:
  def authorize(payment: Payment): F[Authorization]
```

Programs

- Programs chain capabilities into a pipeline
- Think water pipes, air ducts, and circuits
- The topology matters: how the pieces connect is the design

```scala
final class Cart[F[_]: MonadThrow](
    orders: Orders[F],
    payments: Payments[F]
):
  def checkout(command: CreateOrder): F[OrderId] =
    for {
      orderId <- orders.create(command)
      _       <- payments.authorize(command.payment)
    } yield orderId

  ...
```

- **This is analogous to a work order.**

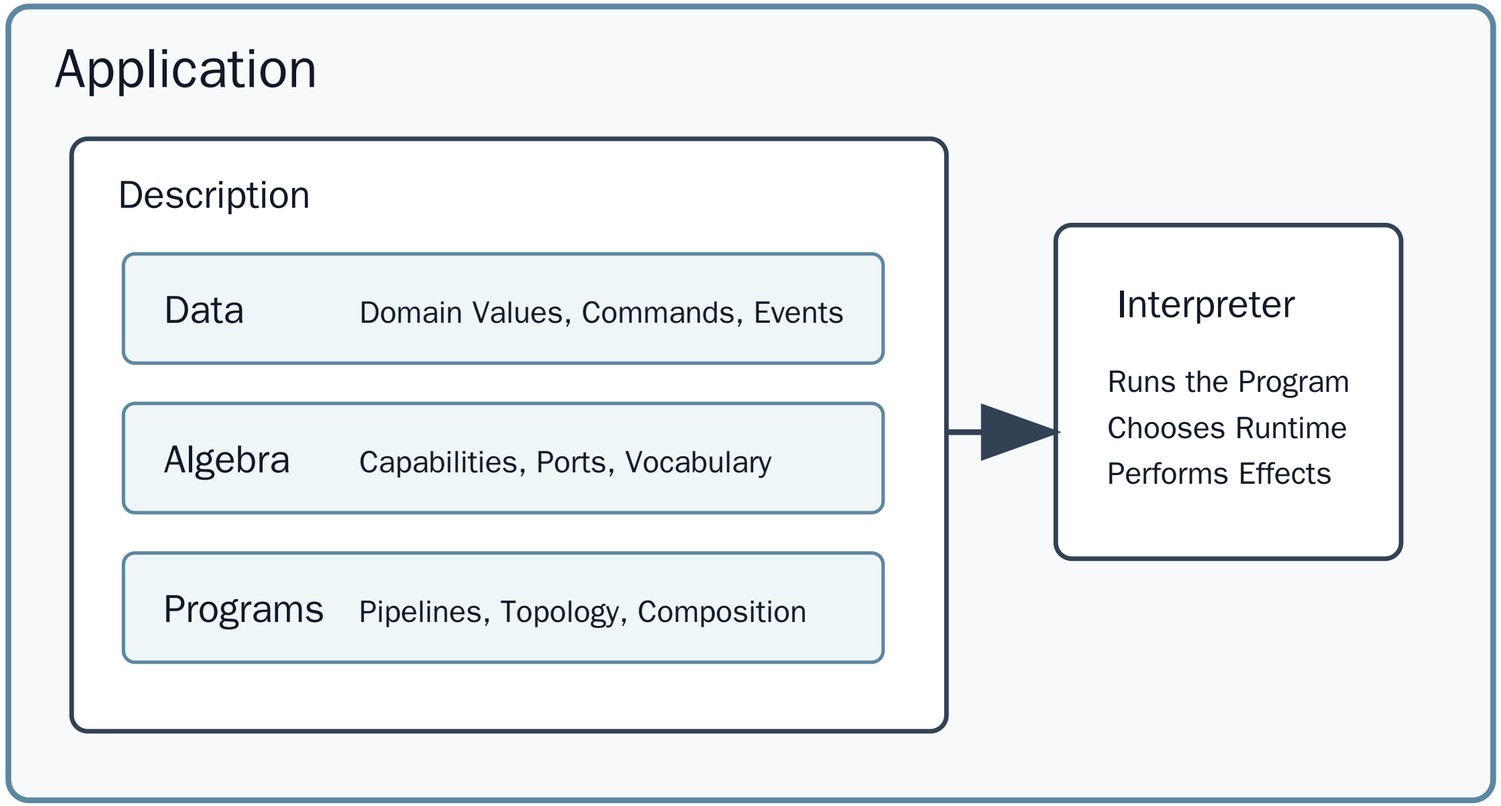
Interpreters

- The interpreter is the part that actually runs
- It takes the described program and chooses a real runtime
- Swap interpreters to change behavior: production vs. test, SQL vs. in-memory
- `unsafeRunAsync` takes the program and runs it at the edge

```scala
val program: IO[OrderId] =
  cart.checkout(command)

program.unsafeRunAsync {
  case Right(orderId) => println(orderId)
  case Left(error)   => error.printStackTrace()
}
```

Application Shape



Why Effects Exist

- Effects isolate impurity at the boundary: the rest of the program stays pure
- Push I/O to the edges; keep the business logic functional and testable

A Note on `@Transactional`

- Java enterprise applications often use annotations or interceptors for transactions
- Spring and Quarkus both support this style, though details differ
- AOP/interceptors wrap a method call with infrastructure behavior

Annotation Boundary

- In annotation-driven Java, the transaction boundary can be implicit infrastructure
- That is useful, but it is not visible in the return type

```java
@Transactional
OrderId createOrder(CreateOrder command) {
    ...
}
```

Interpreter Boundary

- In doobie/http4s, database programs can be values
- `.transact(xa)` interprets `ConnectionIO[A]` into `IO[A]`
- The transaction boundary is explicit at the composition boundary

```scala
val program: ConnectionIO[OrderId] =
  createOrder(command)

val result: IO[OrderId] =
  program.transact(xa)
```

- The unsafe run happens at the outside edge

```scala
result.unsafeRunSync()
```

Demo: HTTP4s and Transactions

Let’s look at a small HTTP4s application and where the transaction boundary is applied.

Conclusion

Java Is Enough to Start

- Java does not have every feature Scala or Kotlin has
- You can still move toward a functional design goal today
- The goal is better code, not language purity
- Experiment with Scala, Kotlin – it’s not scary

What Gets Better

- Readability: smaller methods with clearer inputs and outputs
- Maintenance: fewer hidden effects and fewer surprise mutations
- Testing: more behavior can be exercised without infrastructure
- Aesthetic: code starts to show the shape of the problem

Keep the Direction

- Prefer values to mutation
- Prefer expressions to statements
- Prefer explicit results to hidden control flow
- Prefer separating description from execution

Be Ready for More

- Languages and libraries keep adding functional features
- Java keeps moving, even if it moves carefully
- When stronger abstractions arrive, the mental model will already be familiar

Final Thought

- Thinking functionally is a design discipline
- The syntax helps, but the mindset matters more
- Start with Java, and let better habits carry forward